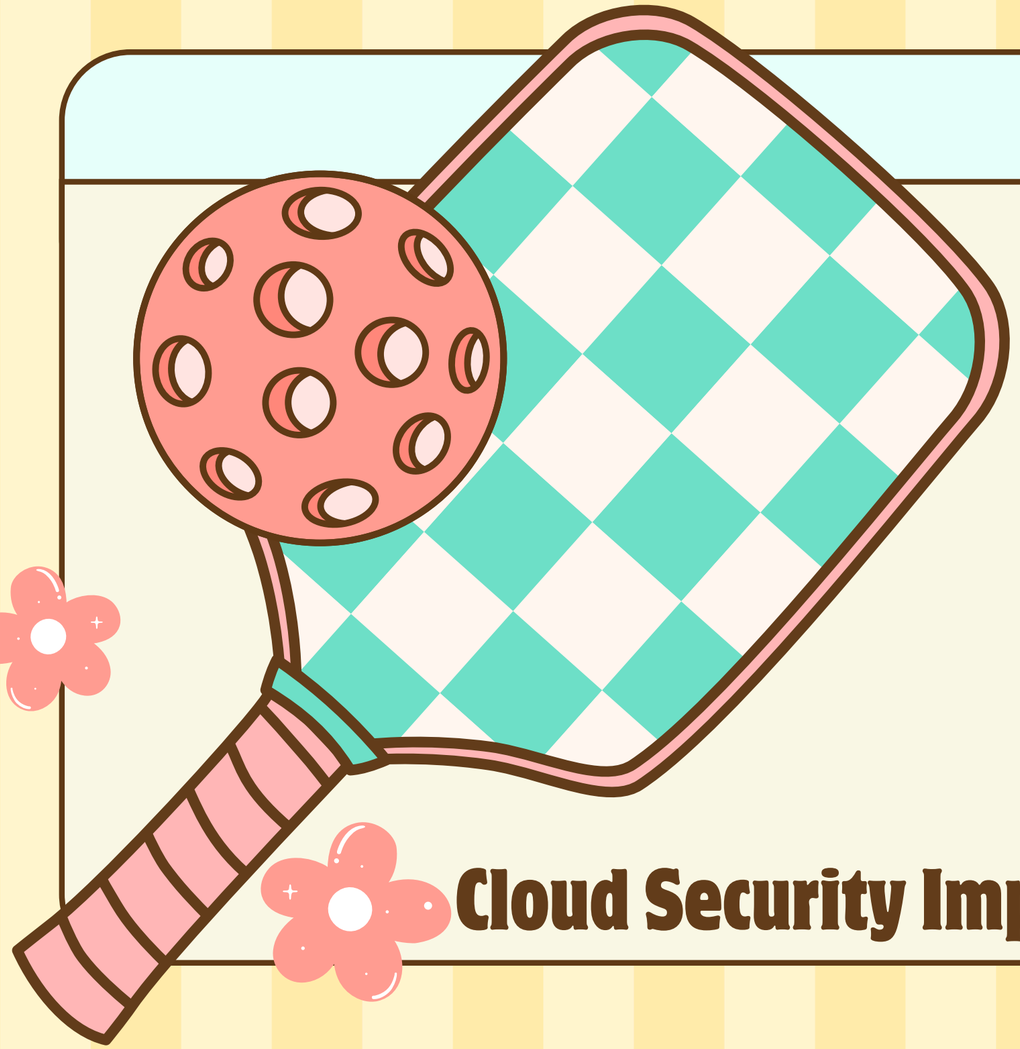
CLOUD SECURITY MINI-PROJECT



# Secure Pickleball Court Management System

## Cloud Security Implementation & Best Practices

Harman Jassal(24070122073), Jalaj Maheshwari(24070122078),
Jatin Rupial(24070122081)

# Why this demo works

**A familiar booking workflow exposes the real security decisions behind cloud deployment.**

THE PROBLEM

## Simple app.
## Real attack surface.

Bookings, admin actions and file uploads create identity, validation, storage and network risks—without requiring a complex business domain.

EVALUATION OBJECTIVES

1. Deploy a live cloud application
2. Enforce least-privilege access
3. Keep reports encrypted and private
4. Validate inputs on the server

# Technology stack

CLIENT

## React

Court availability & booking

EDGE

## Vercel

Frontend + API hosting

COMPUTE

## EC2

Separate deployment pathway

STORAGE

## S3

Private tournament reports

ACCESS

## IAM

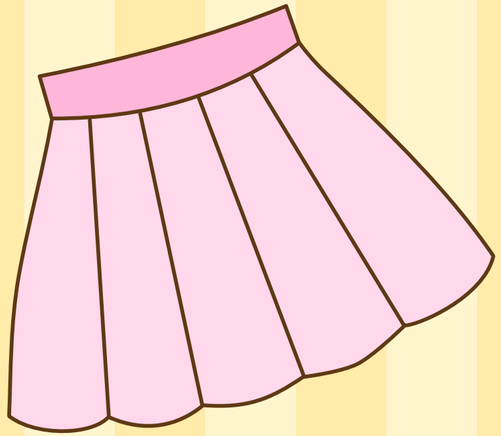
Users, roles & policies

API

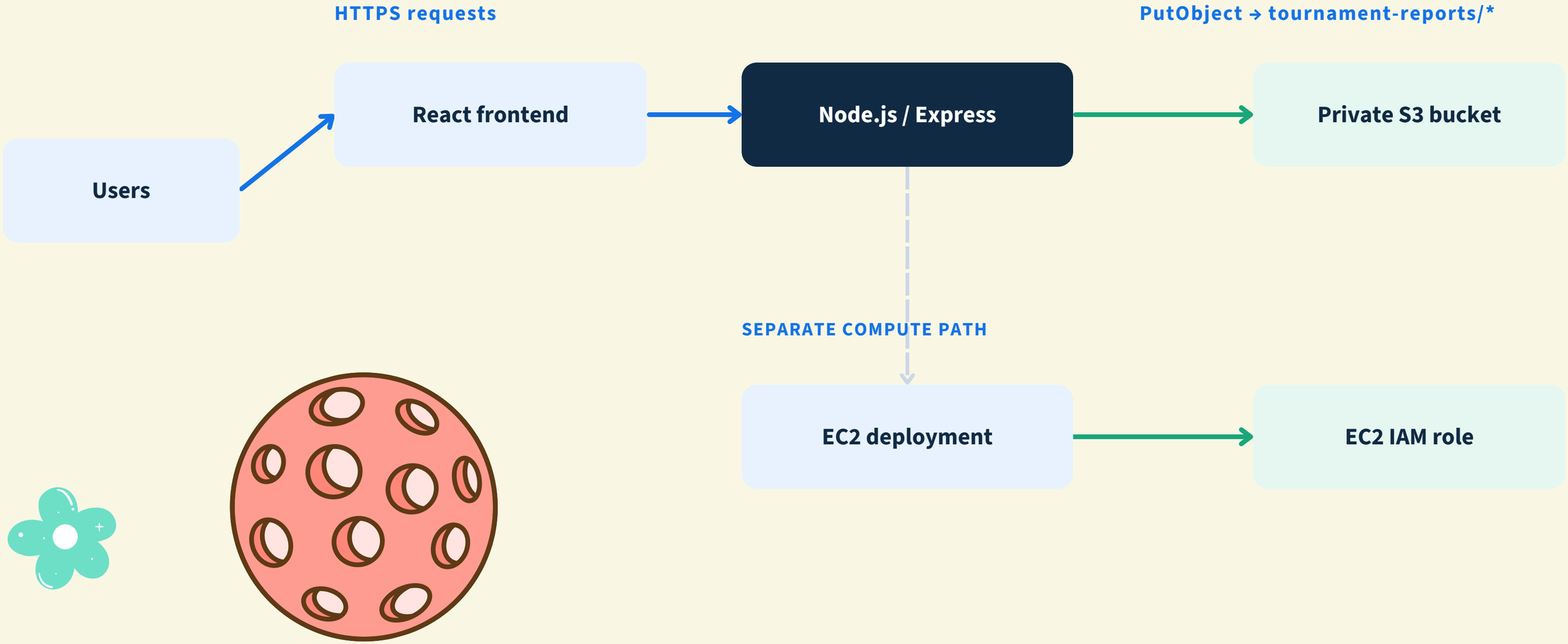
## Node.js / Express

Validation & protected routes





# System architecture



# Application workflow

Check court availability → Submit a booking → Admin authentication → **Upload tournament report** → Encrypted private S3 object



Trust increases →

Every state-changing action is re-validated by the backend.

# IAM & least privilege

Two workloads. Two identities. No credentials embedded in source code.

VERCEL PATH

## Dedicated IAM user

Custom policy allows only s3:PutObject to one prefix.

EC2 PATH

## Instance role

Temporary role credentials—no access keys on the server.

**Allow s3:PutObject**
**Resource …/tournament-reports/***
**Deny by omission everything else**

# S3 data protection

### SSE-S3 encryption

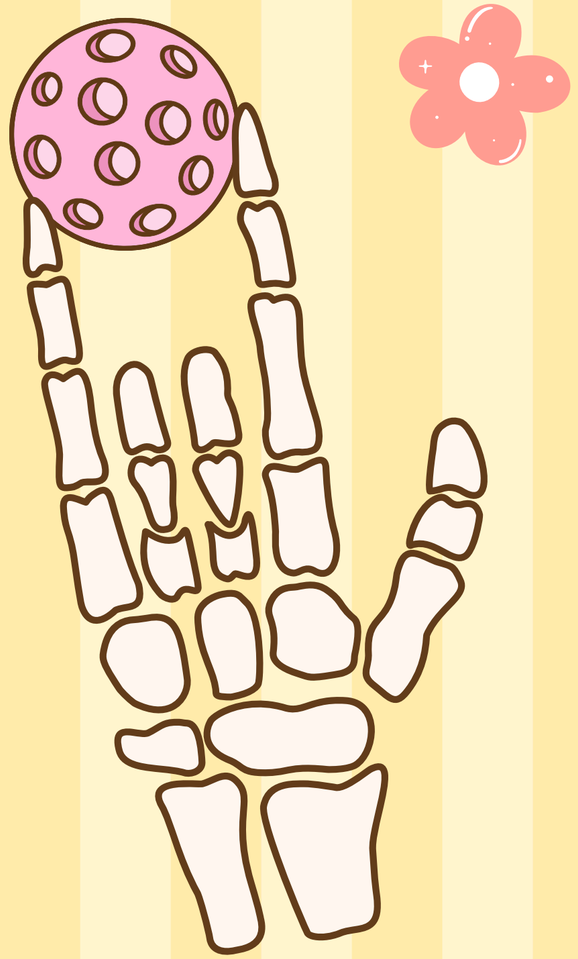
Objects encrypted at rest automatically.

### Block Public Access

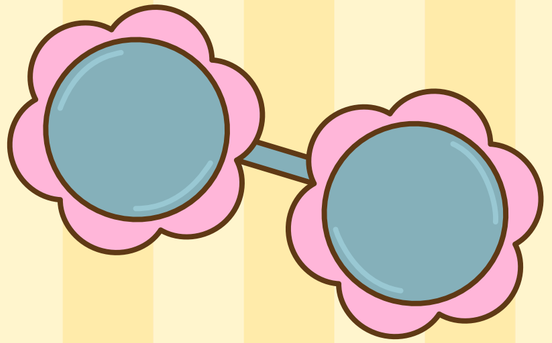
All four public-access controls enabled.

### Bucket owner enforced

Object ownership centralized; ACLs disabled.



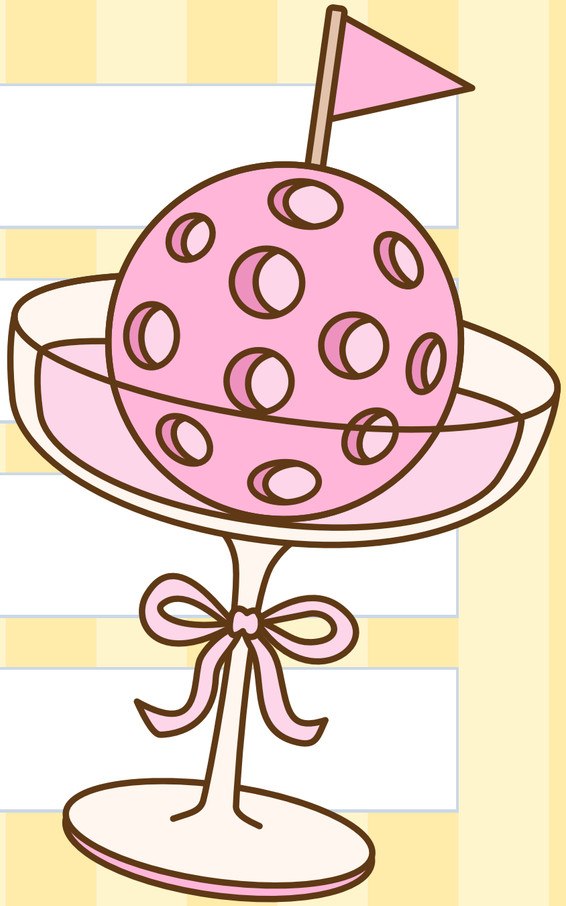
# Defense in depth



| | | |
|---|---|---|
| 01 | **HTTPS** | Encrypted traffic in transit |
| 02 | **Server validation** | Never trust browser input |
| 03 | **Protected admin routes** | Authentication before privileged actions |
| 04 | **Restricted EC2 security group** | Only required inbound access |
| 05 | **Session Manager** | Administrative access without open SSH |

# Implementation & test evidence

| | | |
|---|---|---|
| LIVE | **S3 report upload** | Succeeded; object visible in bucket |
| CONFIG | **S3 protection** | Encryption, public access and ownership verified |
| LOCAL | **Automated tests** | 31 tests reported passing |
| SCOPE | **Hosted paths** | Not all hosted paths were tested |



# Live demo sequence





01

**Open frontend**

02

**Book a court**

03

**Admin login**

04

**Upload report**

05

**Find S3 object**

06

**Prove privacy + IAM**





# From demonstrator to production



TODAY

## Secure foundations

Private encrypted storage

Least-privilege identities

Server-side validation

Restricted network access

NEXT

## Production hardening

Rotate and centrally manage credentials

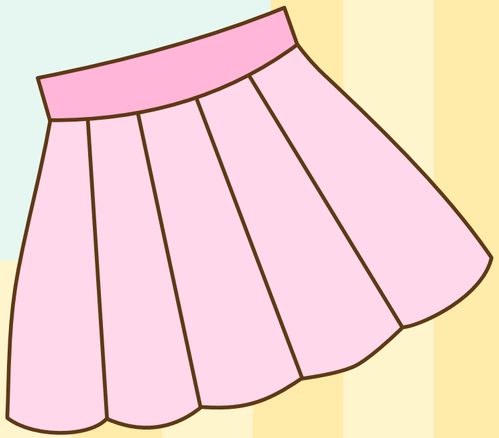
Monitor and alert on audit logs

Strengthen admin authentication

Formalize hosted end-to-end tests

Add report retrieval if required





# Security is the architecture

1

**ENCRYPT**

at rest + in transit

2

**LIMIT**

identity permissions

3

**PRIVATE**

object storage

4

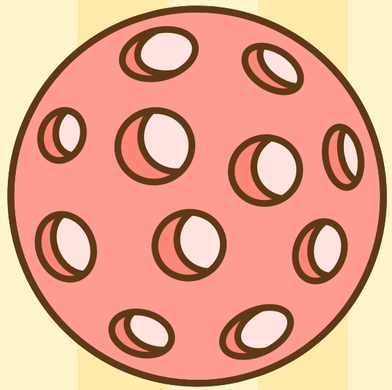
**VALIDATE**

on the server

5

**RESTRICT**

network access



**Design for least privilege. Prove every control.**

# THANKYOU

